CSE STUDY HUB

C++ Programming

UNIT 1 · Advanced Object-Oriented Programming

What this unit is about

This unit builds understanding of **Advanced Object-Oriented Programming**. These notes are designed for quick learning and exam revision: learn the meaning, understand the working, practise an example, and finish with comparisons and key terms.

Syllabus coverage checklist

1. OOP recap: classes, objects, encapsulation, inheritance and polymorphism

2. Compile-time vs runtime polymorphism

3. Virtual functions, overriding and dynamic binding

4. Base pointers/references and virtual dispatch

5. Abstract classes and pure virtual functions (= 0)

6. Virtual destructors and safe polymorphic deletion

7. SOLID principles: SRP, OCP, LSP, ISP and DIP; practical design examples

How to study: Read each topic once for meaning, underline definitions and formulas, reproduce diagrams/steps without looking, then answer the revision questions on Page 5.

Exam answer pattern

For a 5-mark answer: (1) definition, (2) labelled diagram or formula, (3) working/components in steps, (4) example/application, and (5) one advantage, limitation or comparison.

UNIT 1 NOTES · PART 1

OOP recap: classes, objects, encapsulation, inheritance and polymorphism

OOP recap: classes, objects, encapsulation, inheritance and polymorphism is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Compile-time vs runtime polymorphism

Compare the alternatives in Compile-time vs runtime polymorphism using the same criteria: purpose, working method, performance, cost/complexity, security or reliability, and suitable use case. End with a short conclusion stating when each option is preferable.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Virtual functions, overriding and dynamic binding

A virtual function enables runtime dispatch. When a base pointer/reference refers to a derived object, the overridden derived implementation is selected at runtime. A virtual destructor is important for safe deletion through a base pointer.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Advanced Object-Oriented Programming, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 1 NOTES · PART 2

Base pointers/references and virtual dispatch

Base pointers/references and virtual dispatch is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Abstract classes and pure virtual functions (= 0)

A virtual function enables runtime dispatch. When a base pointer/reference refers to a derived object, the overridden derived implementation is selected at runtime. A virtual destructor is important for safe deletion through a base pointer.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Advanced Object-Oriented Programming, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 1 NOTES · PART 3

Virtual destructors and safe polymorphic deletion

Virtual destructors and safe polymorphic deletion is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

SOLID principles: SRP, OCP, LSP, ISP and DIP; practical design examples

SOLID principles: SRP, OCP, LSP, ISP and DIP; practical design examples is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Advanced Object-Oriented Programming, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

QUICK REVISION & EXAM PRACTICE

Key takeaways

• **OOP recap: classes, objects, encapsulation, inheritance and polymorphism** — know the definition, core working and one example.

• **Compile-time vs runtime polymorphism** — know the definition, core working and one example.

• **Virtual functions, overriding and dynamic binding** — know the definition, core working and one example.

• **Base pointers/references and virtual dispatch** — know the definition, core working and one example.

• **Abstract classes and pure virtual functions (= 0)** — know the definition, core working and one example.

• **Virtual destructors and safe polymorphic deletion** — know the definition, core working and one example.

• **SOLID principles: SRP, OCP, LSP, ISP and DIP; practical design examples** — know the definition, core working and one example.

Worked-answer framework

Scenario: A question asks you to explain or apply a concept from Advanced Object-Oriented Programming. **Step 1:** identify the relevant term and define it. **Step 2:** write the formula, components or ordered process. **Step 3:** apply it to the given data/scenario. **Step 4:** state the result and justify it. **Step 5:** mention one limitation or precaution.

Self-test questions

Q1. Define OOP recap: classes, objects, encapsulation, inheritance and polymorphism and explain its importance.

Q2. Explain the working/steps involved in Compile-time vs runtime polymorphism.

Q3. Differentiate between two related concepts from Advanced Object-Oriented Programming.

Q4. Give one practical application of SOLID principles: SRP, OCP, LSP, ISP and DIP; practical design examples and discuss one limitation.

Q5. Write a structured 5-mark answer covering the main ideas of Advanced Object-Oriented Programming.

Final checklist: I can define every topic · explain the process without notes · reproduce formulas/diagrams · solve one example · compare related concepts.